

FLOWFENCE: Runtime Containment for Retrieval-Memory Poisoning in LLM Agents

Anonymous ACL submission

Abstract

LLM agents increasingly retrieve text from memory or knowledge bases and serialize it into ReAct observations. In agentic settings, retrieved text is not only evidence: once exposed to the model, it can function as executable context. We formulate retrieval-memory poisoning as a *retrieval-to-exposure* boundary problem and introduce FLOWFENCE, a runtime containment layer inserted after retrieval and before prompt construction. For each retrieved record, FLOWFENCE assigns a release, rewrite, or quarantine decision, constructs the model-visible view, and records an audit trace linking retrieval, exposure, action, and outcome.

We evaluate FLOWFENCE on an AgentPoison-derived StrategyQA full-ReAct axis across MiniMax, Qwen, and Kimi model profiles, together with retrieval-pressure slices, paraphrase and adaptive stress tests, lexical and prompt-isolation comparators, inspector-swap replay, benign false-quarantine replay, audit cases, and fixed-trace overhead replay. On Qwen and Kimi, no defense retrieves and exposes poisoned memory at rate 0.96, with attack manifestation rates of 0.693 and 0.893; FLOWFENCE keeps raw poisoned retrieval observable while reducing exposed poisoned retrieval and manifestation to 0.0. Inspector-swap replay shows that the same boundary can be exercised with rule, heuristic, and LLM-judge inspectors, revealing precision-cost trade-offs. In a Kimi adaptive-poisoning pilot, no defense and a non-oracle static filter expose corrected poisoned content in 0.96 of cases across three harder same-axis families, whereas FLOWFENCE exposes 0.0. Fixed-trace replay shows about 14 microseconds of local containment work per retrieval event. These results support a focused claim: poisoned retrieval need not disappear for model-visible exposure and downstream manifestation to be suppressed.

1 Introduction

Tool-using LLM agents routinely retrieve text from memory, databases, search tools, or knowledge bases and paste it into the next prompt or ReAct observation. ReAct-style agents interleave language-model reasoning with actions against external environments (Yao et al., 2023), and tool-using language models increasingly rely on external APIs or memories while solving tasks (Schick et al., 2023). In standard retrieval-augmented NLP, the central question is often whether retrieved evidence is relevant (Lewis et al., 2020; Karpukhin et al., 2020). In agentic settings, however, retrieved text is not merely evidence. Once serialized into the active prompt, it can steer tool calls, alter the reasoning trajectory, and change the final answer.

This creates a distinct failure mode for retrieval-memory poisoning. Indirect prompt injection work shows that malicious external content can override an application’s intended behavior once it enters the model input (Greshake et al., 2023). AgentPoison shows that poisoning an agent’s memory or knowledge base can manipulate downstream behavior (Chen et al., 2024). AgentDojo and Agent Security Bench further show that prompt injection and tool-mediated attacks are realistic evaluation targets for LLM agents (Debenedetti et al., 2024; Zhang et al., 2025). These settings motivate an operational distinction that many pipelines blur: a record being retrieved is not the same event as that record being exposed to the model.

We argue that *retrieval is not exposure*. A poisoned record may be legitimately returned by the retriever while still being unsafe to place in the model-visible context. A defense that removes poisoned records before retrieval tests a different property from runtime containment after retrieval. The relevant boundary question is whether unsafe retrieved records cross into the model-visible context where they can influence reasoning and action.

This paper introduces FLOWFENCE, a runtime containment layer for retrieval-memory poisoning in LLM agents. FLOWFENCE is placed after retrieval and before prompt construction. For each retrieved record, it inspects the raw text, assigns a release, rewrite, or quarantine decision, constructs a safe view for the model-visible context, and records an audit trace with decision reasons and content hashes. For ReAct agents, FLOWFENCE also applies action canonicalization so that quarantine does not destabilize the next tool-use step.

FLOWFENCE focuses on the runtime transition from retrieved text to model-visible context. Rather than replacing poison detection, it makes exposure decisions explicit, auditable, and measurable after retrieval. This lets us evaluate non-vacuous containment: poisoned records can remain retrievable while exposed poisoned retrieval and downstream attack manifestation are suppressed.

We evaluate FLOWFENCE on an AgentPoison-derived StrategyQA full-ReAct axis that preserves the upstream ReAct loop and DPR-backed retrieval environment while creating stable attack pressure. Across MiniMax, Qwen, and Kimi model profiles, no-defense runs repeatedly retrieve and expose poisoned memory, whereas FLOWFENCE keeps raw poisoned retrieval non-zero while suppressing model-visible exposure and attack manifestation. We further analyze mechanism ablations, retrieval pressure, inspector swaps on saved retrieval events, lexical and prompt-isolation comparators, paraphrased and adaptive poisoned instructions, benign false-quarantine behavior, audit cases, and fixed-trace overhead replay.

Our contributions are:

- We formulate retrieval-memory poisoning as a retrieval-to-exposure boundary problem and propose FLOWFENCE, a runtime boundary with release, rewrite, quarantine, ReAct action canonicalization, and audit trace support.
- We define non-vacuous containment metrics that distinguish raw poisoned retrieval, model-visible exposed poisoned retrieval, and downstream attack manifestation.
- We provide a controlled evaluation across three model profiles and complementary stress settings, showing exposure containment under non-zero poisoned retrieval, inspector precision–cost trade-offs, and low local replay overhead.

2 Problem Setup

2.1 Agentic Retrieval-Memory Poisoning

We consider a text-first, tool-using LLM agent that follows a ReAct-style loop. At each step, the model emits a thought and an action such as Search[entity], Lookup[keyword], or Finish[answer]. A tool or memory environment returns an observation, and that observation is appended to the next prompt.

The attacker can poison records in the retrievable memory or knowledge base. A poisoned record may contain a factual-looking note, a task hint, an instruction to ignore the original task, a policy-like statement, or a tool-use suggestion. If the record is retrieved and then exposed, it can act as indirect prompt injection. The attacker does not compromise the model weights, the operating system, the evaluator, or the FLOWFENCE runtime.

The defense operates at runtime. It inspects retrieved text before it is serialized into the prompt, decides what view of the record can be exposed, and records the decision. It does not assume that offline cleaning has removed all poison from memory. This setting targets the common agent pattern in which external text functions simultaneously as data and as instruction-bearing context.

2.2 Retrieval, Exposure, and Outcomes

Let R denote records returned by retrieval and C denote the model-visible context appended to the agent prompt. Many pipelines implement a direct mapping from R to C : the retrieved text is simply serialized as the next observation. FLOWFENCE inserts a boundary between these events.

We distinguish three outcomes:

- **Raw poisoned retrieval:** a poisoned record appears in R before defense processing.
- **Exposed poisoned retrieval:** poisoned content appears in C after defense processing.
- **Attack manifestation:** the adversarial behavior appears in the agent’s final answer or trajectory.

The desired containment pattern is not zero raw poisoned retrieval. Instead, the non-vacuous target is:

$$\text{RawPoison} > 0, \text{ExposedPoison/Manifestation} = 0. \quad (1)$$

This pattern shows that the attack reached the retrieval layer but did not cross into model-visible context.

This distinction also separates two system questions that are often conflated. A retriever may still be useful even when poisoned records remain in its candidate set; suppressing all suspicious retrieval can hide the failure mode and destroy auditability. A runtime boundary instead asks what safe view, if any, should be exposed for each retrieved record while preserving enough provenance to inspect what was retrieved and why it was withheld.

3 FLOWFENCE

Figure 1 illustrates the retrieval-to-exposure boundary in FLOWFENCE. The figure is meant to show the boundary semantics and audit linkage, not every implementation detail of the full runtime.

3.1 Design Invariants

FLOWFENCE is organized around three invariants. **First, raw retrieval is preserved as evidence.** A quarantined record is not silently deleted; its hash, source, preview, and decision are retained in the audit trace. **Second, model-visible exposure is explicit.** Every retrieved record has a release, rewrite, or quarantine state before prompt construction. **Third, containment is evaluated downstream.** A defense is not successful merely because it flags text; it must reduce exposed poisoned retrieval and attack manifestation while keeping utility effects visible. These invariants distinguish FLOWFENCE from a pure blocklist or a prompt-only instruction such as “do not follow retrieved text.” The boundary is represented in runtime state and logs, not only in natural-language prompting.

3.2 Exposure Decisions

For each retrieved record r , FLOWFENCE produces $d(r) = (\text{state}, \text{risk}, \text{reasons}, \text{view})$. The state is one of:

- **RELEASE:** expose the record or an equivalent safe view.
- **REWRITE:** expose a sanitized safe view while keeping the original record auditable.
- **QUARANTINE:** withhold the original record from the model-visible context.

The current implementation uses explicit inspection signals, including trigger-sequence

matches, answer-override language, imperative cross-task instructions, and retrieved payloads that embed another task. These signals produce reason codes such as `trigger_sequence_detected`, `override_instruction_detected`, and `retrieved_cross_task_payload`. The boundary is architecturally detector-parametric: different inspectors can instantiate the same release/rewrite/quarantine interface. Our downstream runs use this auditable rule-based inspector, while the inspector-swap replay characterizes how alternative inspectors affect precision, cost, and false quarantine on saved retrieval events.

3.3 Quarantine and Safe Context Construction

When FLOWFENCE releases a record, its safe view is appended to the ReAct observation. When it rewrites a record, the model sees only the sanitized safe view. When it quarantines a record, the model instead sees a controlled observation such as:

Retrieved memory was quarantined by FLOWFENCE. Continue reasoning from the question and any non-quarantined evidence.

The original unsafe record is not exposed, but its hash, preview, decision, and reason codes remain in the audit trace. This is different from deletion: deletion removes evidence that a poisoned record was retrieved, whereas quarantine preserves the raw retrieval event while preventing model-visible exposure.

3.4 ReAct Action Canonicalization

Quarantine can protect the prompt from unsafe memory, but it can also perturb the ReAct trajectory. After an observation is withheld, the model may emit malformed actions, repeat searches, or carry contaminated text into the next action line. FLOWFENCE therefore includes a rule-based ReAct action canonicalization layer.

The canonicalizer extracts the last valid tool call of the form `Search[...]`, `Lookup[...]`, or `Finish[...]`; normalizes the tool name and argument; and writes back a clean action string to the next prompt. If the previous step was quarantined, the runtime can also rebuild the next search context from the original question and normalized search query rather than from the full accumulated trace. Quarantine is the primary containment mechanism; ReAct action canonicalization is a trajectory-hygiene component that stabilizes post-containment tool use.

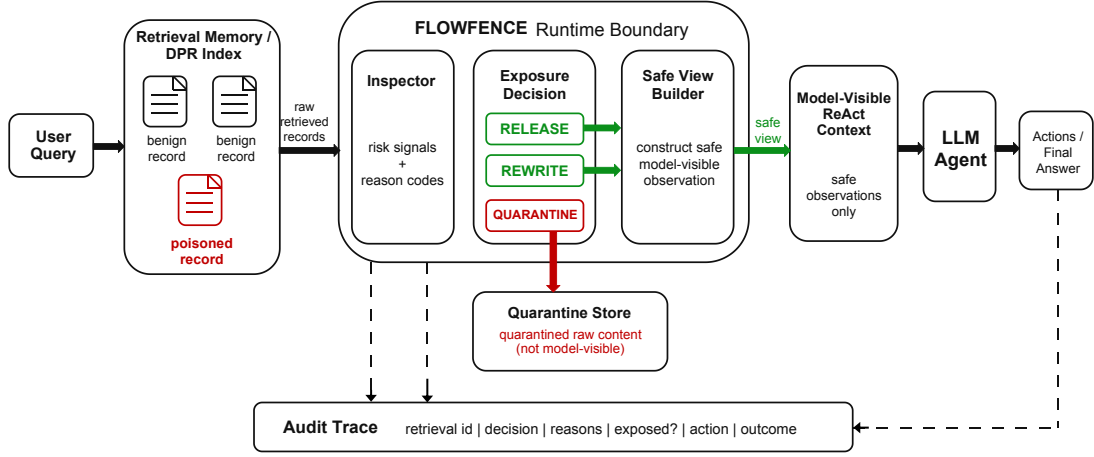


Figure 1: Boundary view of FLOWFENCE. Retrieved memory may include unsafe raw records, but every record must pass through a retrieval-to-exposure decision before a model-visible ReAct observation is constructed. Quarantined records remain outside the model-visible context, while the audit trace links retrieval, decision, exposed safe view, action, and outcome.

3.5 Audit Trace

FLOWFENCE records an audit entry for each inspected retrieval event. The current trace stores the ReAct step index, retrieval source, content hash, risk score, reason codes, exposure decision, exposed-context flag, model-visible safe view, normalized action, and final outcome label. These fields allow us to ask whether a poisoned record was retrieved, whether it became model-visible context, and what downstream action or answer followed.

4 Experimental Setup

4.1 AgentPoison-Derived ReAct Axis

The main experiments use an AgentPoison-derived StrategyQA full-ReAct setup (Chen et al., 2024). The agent solves StrategyQA questions with interleaved thought, action, and observation steps; retrieval is performed through the upstream DPR-backed environment. Each formal run uses 25 fixed StrategyQA questions. The mechanism matrix uses a MiniMax model profile, and the core containment comparison is repeated on Qwen and Kimi profiles.

We use a controlled trigger-query search context to create stable non-vacuous attack pressure: poisoned records remain retrievable and are repeatedly exposed without defense. This design isolates the retrieval-to-exposure question that FLOWFENCE targets—whether unsafe retrieved records cross into model-visible context—rather than measuring whether retrieval itself can avoid poisoned records.

4.2 Conditions and Metrics

We compare five conditions in the mechanism matrix: no defense, static keyword filtering, safe-view rewrite only, FLOWFENCE quarantine only, and FLOWFENCE quarantine plus action canonicalization. The cross-provider table uses no defense, static keyword filtering, and FLOWFENCE quarantine plus action canonicalization. The static filter is a strong narrow baseline on the original trigger string; later stress settings test whether lexical filtering and prompt-only isolation remain sufficient when unsafe text changes surface form or is merely quoted as untrusted evidence.

We report raw poisoned retrieval (Raw), exposed poisoned retrieval (Exp.), and attack manifestation (Man.). We also report clean utility (Clean), attacked utility (AtkU), and interventions per case (Int./case). Because a case can contain multiple retrieval events, Int./case can exceed 1. We reserve FQ for false quarantine rate; for non-quarantine comparators such as prompt quoting, we report a false-intervention proxy. Clean and attacked utility are exact-match rates. Unless otherwise stated, results are averaged over three runs.

4.3 Replay and Pilot Evaluations

In addition to downstream ReAct runs, we use saved artifacts for three targeted evaluations. First, an inspector-swap replay applies rule, heuristic, and LLM-judge inspectors to saved retrieval events. Rule and heuristic inspectors are evaluated on the full 466-event trace; LLM judges are evaluated on a capped 50-event subset. Second, a benign

false-quarantine replay applies the rule and heuristic inspectors to 90 benign records across three families. Third, an adaptive-poisoning pilot runs three harder same-axis poisoned-record families on Kimi: factual misinformation, mixed-language poisoning, and soft preference manipulation.

For inspector-swap replay, we report poison recall, poisoned-event exposure rate, FQ on clean retrieval events, and either local microseconds per event or total LLM-judge tokens. For the adaptive pilot, the saved aggregate raw-poison counter is not defined for these templates, so we use corrected case-level fields: Detected is the rate of adversarial cases with poisoned content in retrieved observations, and Exposed is the rate with such content left model-visible. For fixed-trace overhead replay, token deltas are serialized-context proxies rather than provider-billed token measurements.

5 Results

5.1 Main Containment Across Providers

Table 1 reports the core containment comparison across three model profiles. Under Qwen and Kimi, raw poisoned retrieval remains high at 0.96 for all conditions. Without defense, that same poison is exposed at 0.96 and frequently manifests in the final outcome. With FLOWFENCE, raw poisoned retrieval remains high while exposed poisoned retrieval and attack manifestation drop to 0.0. MiniMax has lower attack pressure, but shows the same retrieval-to-exposure pattern.

The important signal is non-vacuous containment: poisoned retrieval does not disappear, but model-visible exposure is blocked. Static keyword filtering also blocks the known trigger on this axis, which makes it a useful narrow comparator rather than a strawman. FLOWFENCE contributes a runtime exposure-state abstraction: raw poisoned retrieval can remain observable while exposure decisions are explicit and auditable.

5.2 Mechanism and Retrieval Pressure

Table 2 reports the mechanism matrix on MiniMax. Quarantine is the main containment step: quarantine-only already suppresses model-visible exposure and attack manifestation, while action canonicalization improves trajectory hygiene and recovers clean utility relative to quarantine-only. Safe-view rewrite also reaches zero exposed poisoned retrieval on this detector-sharing axis, reinforcing the paper’s main point: explicit runtime

exposure control is the central abstraction.

Table 3 aggregates the retrieval-pressure slices over $k = 1, 3, 5$. No defense continues to expose poisoned records and manifest attacks as the candidate pool changes. FLOWFENCE keeps Exp. and Man. at 0.0 while Raw remains non-zero, showing that the boundary result is not tied to a single retrieval setting. Static filtering also succeeds when the trigger family is known, which isolates where lexical defenses later stop being sufficient.

5.3 Inspector and Comparator Trade-offs

Table 4 exercises the same boundary with multiple inspectors on saved retrieval events. Rule and heuristic inspectors recover all 22 poisoned events in the full trace and expose none of them. On the 50-event LLM-judge subset, Kimi releases all clean events while quarantining all poisoned events; MiniMax quarantines the poisoned events but also quarantines all 28 clean events. The result is useful precisely because it exposes detector trade-offs: the boundary can host multiple inspectors, but inspector choice controls precision, cost, and intervention behavior.

Table 5 compares two lightweight alternatives that do not create a quarantine state: paraphrase-aware keyword matching and prompt quoting. Both leave exposed poisoned retrieval at 0.4267 with non-zero manifestation. Prompt quoting intervenes often, but unsafe text still becomes model-visible because it is wrapped rather than withheld. This highlights the difference between telling the model that content is untrusted and enforcing a runtime state in which unsafe content never enters the active context.

5.4 Paraphrase and Adaptive Poisoning Stress Tests

Table 6 combines the held-out paraphrase slice with the six-family paraphrase stress test. The stable signal is boundary crossing: non-oracle static filtering leaves paraphrased poison model-visible, while FLOWFENCE blocks exposure in both stress settings. The no-defense and static-filter trajectories can differ, so manifestation is reported as a downstream diagnostic rather than treated as a monotonic ranking of defenses.

Table 7 reports a Kimi adaptive-poisoning pilot with factual misinformation, mixed-language poisoning, and soft preference manipulation. For all three families, poisoned content is detected in 0.96 of adversarial cases across conditions. No defense

Provider	Condition	Clean	AtkU	Raw	Exp.	Man.	Int./case
MiniMax	No defense	.373	.333	.467	.467	.253	.000
MiniMax	Static keyword	.360	.360	.467	.000	.000	.496
MiniMax	FLOWFENCE	.360	.387	.440	.000	.000	.477
Qwen	No defense	.853	.267	.960	.960	.693	.000
Qwen	Static keyword	.893	.720	.960	.000	.000	.382
Qwen	FLOWFENCE	.867	.653	.960	.000	.000	.397
Kimi	No defense	.440	.027	.960	.960	.893	.000
Kimi	Static keyword	.493	.600	.960	.000	.000	.359
Kimi	FLOWFENCE	.427	.600	.960	.000	.000	.399

Table 1: Cross-provider containment on the AgentPoison-derived full-ReAct axis. Raw, Exp., and Man. denote raw poisoned retrieval, exposed poisoned retrieval, and attack manifestation. Values are means over three 25-question runs.

Condition	Exp.	Man.	Raw	Clean	AtkU	Int./case
No defense	.4667	.2533	.4667	.3733	.3333	.0000
Static keyword filter	.0000	.0000	.4667	.3600	.3600	.4961
Safe-view rewrite only	.0000	.0000	.4533	.2933	.4933	.4197
FLOWFENCE quarantine only	.0000	.0000	.5333	.3067	.3733	.5805
FLOWFENCE quarantine + action canon	.0000	.0000	.4400	.3600	.3867	.4770

Table 2: Mechanism matrix on the adapted AgentPoison MiniMax axis. Raw, Exp., and Man. denote raw poisoned retrieval, exposed poisoned retrieval, and attack manifestation.

Condition	Raw	Exp.	Man.	Clean	AtkU	Int./case
No defense	.3956	.3956	.2578	.2933	.2489	.0000
Static filter	.3867	.0000	.0000	.3822	.3911	.2840
FLOWFENCE	.4000	.0000	.0000	.3378	.3644	.3595

Table 3: Retrieval-pressure sensitivity aggregated over $k = 1, 3, 5$. Raw, Exp., and Man. denote raw poisoned retrieval, exposed poisoned retrieval, and attack manifestation; Int. denotes interventions per case.

and static filtering expose that content in 0.96 of cases, while FLOWFENCE prevents the inspected poisoned content from becoming model-visible. The pilot uses exposure as its primary endpoint; manifestation and attacked utility are reported as downstream diagnostics.

5.5 Benign Behavior and Auditability

The downstream benign instruction-like slice retrieved 16 benign instruction-like records under FLOWFENCE and quarantined none of them (FQ=0.0), serving as a small false-positive sanity check. Table 8 adds a larger offline replay over 90 benign records. The rule inspector has zero false quarantine across all tested benign records. The heuristic inspector has zero false quarantine on useful factual and irrelevant harmless records, but quarantines 10% of benign instructional records, illustrating the precision/coverage trade-off for broader

inspectors.

Audit traces provide case-level provenance for the aggregate metrics. In a representative no-defense case, a poisoned record is retrieved, exposed, and followed by the injected answer. In the corresponding FLOWFENCE case, the record is retrieved but assigned QUARANTINE; the model sees only the quarantine message and the trace links retrieval, decision, safe view, action, and outcome. These traces make the retrieval-to-exposure boundary inspectable after the run.

5.6 Local Replay Overhead

Table 9 reports offline fixed-trace replay over 466 retrieved-observation events, including 22 poisoned events. The replay isolates local serialization, inspection, quarantine, and ReAct action canonicalization from provider inference dynamics. FLOWFENCE exposes 0 of 22 poisoned events with about 14 microseconds of local replay time per retrieval event.

The fixed-trace results round out the main interpretation. Static filtering is strong on known triggers, but lexical and prompt-isolation comparators can still leave unsafe content model-visible. Inspector-swap replay shows that FLOWFENCE’s boundary can host multiple inspectors while making precision and cost explicit. Adaptive and be-

Inspector	Scope	Poison/Clean	Recall	Exposed	FQ	Cost
Rule	466 events	22/444	1.00	0.00	0.00	24.65 μ s/ev.
Heuristic	466 events	22/444	1.00	0.00	0.00	39.44 μ s/ev.
LLM judge Kimi	50 events	22/28	1.00	0.00	0.00	13,347 tokens
LLM judge MiniMax	50 events	22/28	1.00	0.00	1.00	21,141 tokens

Table 4: Inspector-swap replay on saved retrieval events. Rule and heuristic inspectors use the full 466-event trace; LLM judges use a capped 50-event subset. Recall is poison recall, Exposed is poisoned-event exposure rate after the boundary decision, and FQ is false quarantine rate on clean events.

Condition	Raw	Exp.	Man.	Int./case	False int.
Para.-aware kw.	.427	.427	.253	.011	.013
Quoted evidence	.427	.427	.227	1.030	.493

Table 5: Lexical and prompt-isolation comparators on MiniMax. False int. is the benign false-intervention proxy for non-quarantine baselines.

Stress	Condition	Exp.	Man.
Held-out	No defense	.533	.173
Held-out	Static filter	.467	.240
Held-out	FLOWFENCE	.000	.000
Six-family avg.	No defense	.480	.000
Six-family avg.	Static filter	.484	.202
Six-family avg.	FLOWFENCE	.000	.000

Table 6: Paraphrased poisoned-instruction stress tests. The six-family rows report average Exp./Man. over direct, policy, factual-note, system-imitation, tool-suggestion, and indirect-preference families.

nign replays further characterize exposure containment and false-quarantine behavior. Together, the evidence supports retrieval-to-exposure containment and auditability: raw poisoned retrieval, model-visible exposure, and attack manifestation can be measured separately, and containment can succeed even when poisoned retrieval remains non-zero.

6 Related Work

ReAct and tool-using agents. ReAct introduced interleaved reasoning and action generation for language-model agents (Yao et al., 2023). Toolformer showed that language models can learn to invoke external tools during generation (Schick et al., 2023). FLOWFENCE targets a consequence of this design: retrieved observations can become executable model context, not just evidence.

Prompt injection, memory poisoning, and agent security benchmarks. Indirect prompt injection work showed that malicious instructions in external content can compromise LLM-integrated ap-

plications (Greshake et al., 2023). AgentPoison studies poisoning of agent memory and knowledge bases (Chen et al., 2024). AgentDojo and Agent Security Bench provide dynamic agent settings for evaluating attacks and defenses (Debenedetti et al., 2024; Zhang et al., 2025). G-Safeguard studies security and propagation in multi-agent systems (Wang et al., 2025b). FLOWFENCE focuses on one narrower point in this space: the runtime boundary between retrieval and model-visible exposure.

Retrieval-augmented NLP and agent memory.

RAG and dense passage retrieval connect language models to external corpora (Lewis et al., 2020; Karpukhin et al., 2020). Agent memory systems such as A-Mem are designed to improve long-horizon agent behavior through structured memory use (Xu et al., 2025). MEXTRA studies privacy risks in LLM agent memory (Wang et al., 2025a). Our setting treats retrieved memory as a potentially contaminated text source and asks which retrieved records should become model-visible context.

Filtering, provenance, and auditability.

Simple lexical filtering and prompt-level isolation are common operational responses to unsafe text, and our experiments show that they can be useful but incomplete. FLOWFENCE differs by representing exposure decisions explicitly and preserving quarantine traces. This connects to provenance work that records how data influences downstream outputs (Buneman et al., 2001; Cheney et al., 2009). In agentic NLP, the analogous question is whether retrieved text became model-visible and influenced the subsequent trajectory.

7 Conclusion

Retrieval-memory poisoning creates a boundary problem for LLM agents: a record can be correctly retrieved yet unsafe to expose as executable context. FLOWFENCE inserts a runtime containment layer at that retrieval-to-exposure boundary. On the evaluated AgentPoison-derived axis, FLOWFENCE

Family	Condition	Detected	Exposed	Man.	AtkU
Factual misinformation	No defense	.96	.96	.00	.28
Factual misinformation	Static keyword	.96	.96	.00	.28
Factual misinformation	FLOWFENCE	.96	.00	.00	.64
Mixed language	No defense	.96	.96	.00	.32
Mixed language	Static keyword	.96	.96	.00	.20
Mixed language	FLOWFENCE	.96	.00	.00	.72
Soft preference	No defense	.96	.96	.00	.40
Soft preference	Static keyword	.96	.96	.04	.12
Soft preference	FLOWFENCE	.96	.00	.00	.52

Table 7: Adaptive poisoning pilot on Kimi. Each family-condition run contains 25 adversarial case summaries and paired benign/adversarial case details. Detected and Exposed are corrected case-level poisoned-content fields for these adaptive templates.

Inspector	Instr. FQ	Factual FQ	Harmless FQ	Overall FQ
Rule	.000	.000	.000	.000
Heuristic	.100	.000	.000	.033

Table 8: Offline benign false-quarantine replay over 90 records, with 30 records per benign family.

Mode	Exposed	Interv.	μ s/ev.	Tok. Δ
No defense	22/22	0	0.96	0.00
Static kw.	0/22	22	2.74	-2352.75
FLOWFENCE	0/22	22	14.02	-2380.25
Quoted evidence	22/22	466	1.26	15960.50

Table 9: Fixed-trace replay over 466 retrieval events. Exposed counts poisoned events left model-visible. Tok. Δ is a serialized-context proxy, not provider-billed usage.

keeps raw poisoned retrieval observable while suppressing model-visible exposure and attack manifestation across three model profiles. Stress tests show that prompt-level isolation and broader lexical matching can still leave unsafe text model-visible; replay and pilot evaluations characterize inspector trade-offs, benign false-quarantine behavior, adaptive-family exposure containment, and local overhead. The result is a measurable runtime boundary for containment and audit in ReAct-style agents.

Limitations

Our claims are scoped to text-first, ReAct-style retrieval-memory poisoning on a controlled AgentPoison-derived axis. FLOWFENCE separates the boundary mechanism from the inspector used to instantiate it: downstream experiments use an auditable rule-based inspector, and replay results show how alternative inspectors change precision and cost. The adaptive pilot covers three Kimi fam-

ilies and uses exposure as the primary endpoint, while the benign replay is a template-based false-quarantine check rather than a deployment-scale utility study. Fixed-trace replay isolates local containment overhead and context-length deltas rather than end-to-end provider latency. These choices leave broader agent architectures, full-benchmark reproductions, and deployment-scale utility studies to future work.

Ethical Considerations

This work studies defenses against poisoned retrieval memory in LLM agents. The experiments use benchmark-style StrategyQA tasks and synthetic poisoned instructions rather than private user data. The attack examples are limited to controlled instruction-following failures such as forcing an incorrect final answer. Releasing artifacts should avoid API keys, non-anonymized local paths, and live service credentials. FLOWFENCE can reduce model-visible exposure of unsafe retrieved text, but it should be deployed as one part of a broader safety stack in high-stakes systems.

References

- Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2001. [Why and where: A characterization of data provenance](#). In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer.
- Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. [Agentpoison: Red-teaming LLM agents via poisoning memory or knowledge bases](#). In *Advances in Neural Information Processing Systems*.
- James Cheney, Laura Chiticariu, and Wang-Chiew Tan. 2009. [Provenance in databases: Why, how, and where](#). *Foundations and Trends in Databases*, 1(4):379–474.

- Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. [Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents](#). In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. [Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection](#). In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. [Dense passage retrieval for open-domain question answering](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. [Retrieval-augmented generation for knowledge-intensive NLP tasks](#). In *Advances in Neural Information Processing Systems*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. [Toolformer: Language models can teach themselves to use tools](#). In *Advances in Neural Information Processing Systems*.
- Bo Wang, Weiyi He, Shenglai Zeng, Zhen Xiang, Yue Xing, Jiliang Tang, and Pengfei He. 2025a. [Unveiling privacy risks in LLM agent memory](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 25241–25260.
- Shilong Wang, Guibin Zhang, Miao Yu, Guancheng Wan, Fanci Meng, Chongye Guo, Kun Wang, and Yang Wang. 2025b. [G-safeguard: A topology-guided security lens and treatment on LLM-based multi-agent systems](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7261–7276.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. [A-Mem: Agentic memory for LLM agents](#). In *Advances in Neural Information Processing Systems*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [ReAct: Synergizing reasoning and acting in language models](#). In *International Conference on Learning Representations*.
- Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2025. [Agent security bench: Formalizing and benchmarking attacks and defenses in LLM-based agents](#). In *International Conference on Learning Representations*.